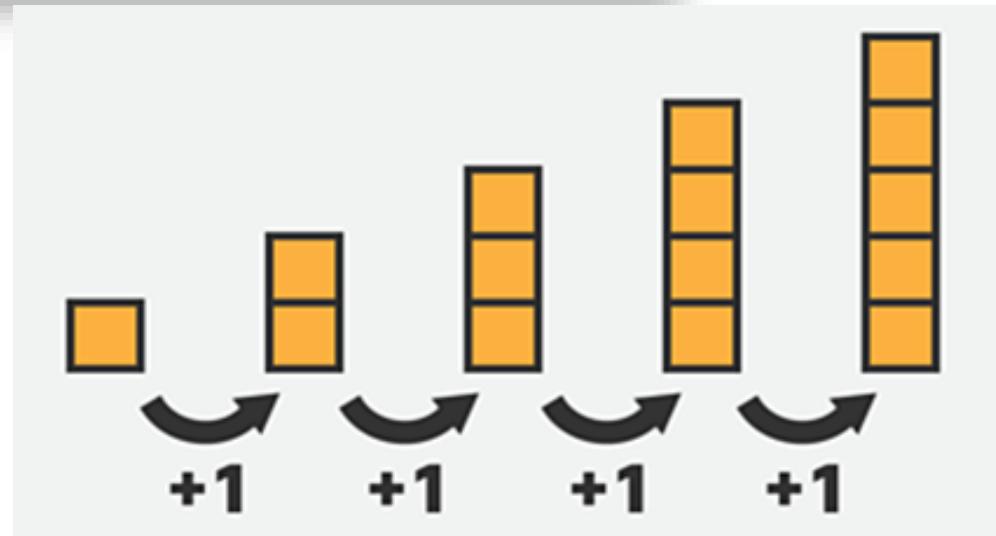
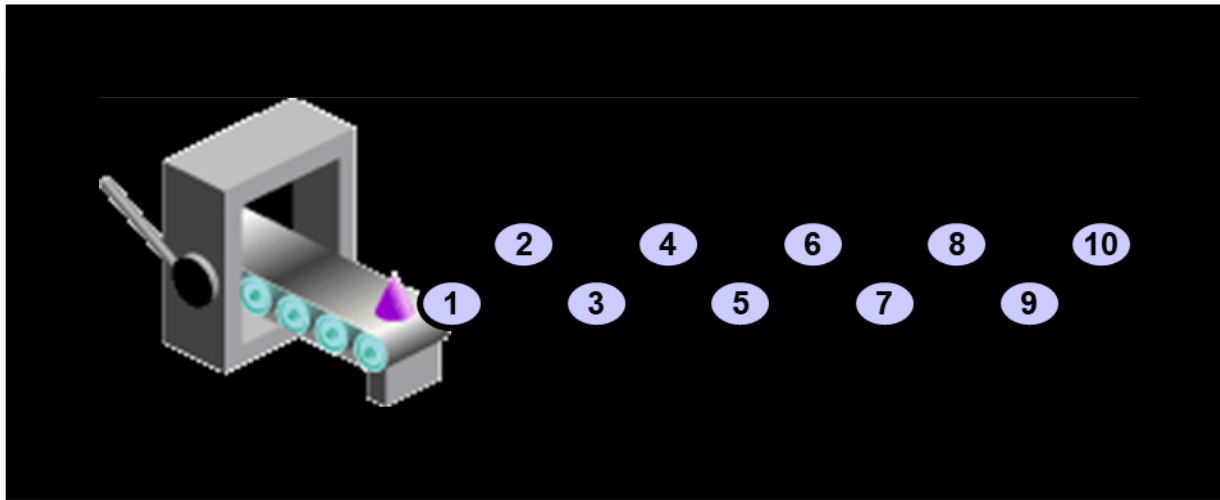


UNIT III: 3.5 Sequences: Creating Sequences, Altering Sequences, Dropping Sequences



Prepared By

Mr. Mohammad Ali

Lecturer (Selection Grade)

M. H. Saboo Siddik Polytechnic, Mumbai

Objectives

After completing this lesson, you should be able to do the following:

- Create, maintain, and use sequences.

SEQUENCES

A sequence:

- can automatically generate sequences of unique integer numbers
- is a type of database object.
- is a sharable object
- can be used to create a primary key values in database tables
- replaces application code
- speeds up the efficiency of accessing sequence values when cached in memory
- can be generated in either ascending or descending order

Creating Sequences – Syntax

The syntax of the CREATE SEQUENCE command is fairly complex because it has numerous optional clauses.

Define a sequence to generate sequential numbers automatically:

```
CREATE SEQUENCE <sequence name>  
[INCREMENT BY <number>]  
[START WITH <start value number>]  
[MAXVALUE <MAXIMUM VLAUE NUMBER> | NOMAXVALUE]  
[MINVALUE <minimum value number> | NOMINVALUE]  
[CYCLE | NOCYCLE]  
[CACHE <number of sequence value to cache> | NOCACHE ]  
[ORDER] | NOORDER];
```

Creating Sequences – Example

```
CREATE SEQUENCE rollno_seq  
INCREMENT BY 1  
START WITH 19401  
MAXVALUE 19432  
MINVALUE 19401  
NOCYCLE  
CACHE 50;
```

Sequence created.

```
SQL> CREATE SEQUENCE rollno_seq  
2 INCREMENT BY 1  
3 START WITH 19401  
4 MAXVALUE 19432  
5 MINVALUE 19401  
6 NOCYCLE  
7 CACHE 50;
```

Sequence created.

Creating Sequences – Example

- Create a sequence named `DEPTNO_SEQ` to be used for the primary key of the `DEPT` table.
- Do not use the `CYCLE` option.

```
CREATE SEQUENCE deptno_seq  
INCREMENT BY 10  
START WITH 50  
MAXVALUE 90  
NOCYCLE  
NOCACHE;
```

Sequence created.

```
SQL> CREATE SEQUENCE deptno_seq  
2 INCREMENT BY 10  
3 START WITH 50  
4 MAXVALUE 90  
5 NOCYCLE  
6 NOCACHE;
```

Sequence created.

Accessing Sequence Values

- Sequence values are generated through the use of two pseudocolumns named
 - **currval**
 - **nextval**
- A pseudocolumn behaves like a table column, but pseudocolumns are not actually stored in a table.
- We can select values from pseudocolumns but cannot perform manipulations on their values.
- The first time we select the **nextval** pseudocolumn, the initial value in the sequence is returned.
- Subsequent selections of the **nextval** pseudocolumn will cause the sequence to increment as specified by the **INCREMENT BY** clause and will return the newly generated sequence value.
- The **currval** pseudocolumn returns the current value of the sequence, which is the value returned by the last reference to **nextval**.

Accessing Sequence Values – Use of **nextval**

```
CREATE TABLE student
(rollno NUMBER(5) CONSTRAINT pk_students PRIMARY KEY,
name varchar2(15), dob date, mobno number(10));
```

The INSERT commands reference the **rollno_seq.nextval** pseudocolumn.

```
INSERT INTO STUDENT VALUES
    (rollno_seq.nextval, 'ANAM', SYSDATE, 9869986998);
INSERT INTO STUDENT VALUES
    (rollno_seq.nextval, 'JUNAID', SYSDATE, 3333333333);
INSERT INTO STUDENT VALUES
    (rollno_seq.nextval, 'HABBAN', SYSDATE, 7000000000);
INSERT INTO STUDENT VALUES
    (rollno_seq.nextval, 'CHETAN', SYSDATE, 1111111111);
INSERT INTO STUDENT VALUES
    (rollno_seq.nextval, 'HUZAIFA', SYSDATE, 6666600000);
```


Accessing Sequence Values – Use of **nextval**

```
SQL> INSERT INTO STUDENT VALUES (rollno_seq.nextval,'ANAM',SYSDATE,9869986998);
1 row created.

SQL> INSERT INTO STUDENT VALUES (rollno_seq.nextval,'JUNAID',SYSDATE,3333333333);
1 row created.

SQL> INSERT INTO STUDENT VALUES (rollno_seq.nextval,'HABBAN',SYSDATE,7000000000);
1 row created.

SQL> INSERT INTO STUDENT VALUES (rollno_seq.nextval,'CHETAN',SYSDATE,1111111111);
1 row created.

SQL> INSERT INTO STUDENT VALUES (rollno_seq.nextval,'HUZAIFA',SYSDATE,6666600000);
1 row created.

SQL> select * from student;
```

ROLLNO	NAME	DOB	MOBNO
19401	ANAM	18-SEP-20	9869986998
19402	JUNAID	18-SEP-20	3333333333
19403	HABBAN	18-SEP-20	7000000000
19404	CHETAN	18-SEP-20	1111111111
19405	HUZAIFA	18-SEP-20	6666600000

Accessing Sequence Values – Use of **currval**

```
SQL> CREATE TABLE fee_detail
  2  (rollno number(5) constraint fk_fee_detail
  3  references student(rollno), tot_fee number(5),
  4  paid_fee number(5), bal_fee AS (tot_fee - paid_fee));
```

Table created.

```
SQL> select * from student;
```

ROLLNO	NAME	DOB	MOBNO
19401	ANAM	18-SEP-20	9869986998
19402	JUNAID	18-SEP-20	3333333333
19403	HABBAN	18-SEP-20	7000000000
19404	CHETAN	18-SEP-20	1111111111
19405	HUZAIFA	18-SEP-20	6666600000

```
SQL> commit;
```

Commit complete.

```
SQL> insert into fee_detail (rollno, tot_fee, paid_fee)
  2  values(rollno_seq.currval, 10000, 7500);
```

1 row created.

```
SQL> select * from fee_detail;
```

ROLLNO	TOT_FEE	PAID_FEE	BAL_FEE
19405	10000	7500	2500

Accessing Sequence Values – Use of DEPTNO_SEQ

```
SQL> SELECT * FROM DEPT;
```

DEPTNO	DNAME	LOC
10	ACCOUNTING	NEW YORK
20	RESEARCH	DALLAS
30	SALES	CHICAGO
40	OPERATIONS	BOSTON

```
SQL> INSERT INTO dept(deptno, dname, loc)
2 VALUES (deptno_seq.NEXTVAL, 'PERSONAL', 'MUMBAI');
```

1 row created.

```
SQL> INSERT INTO dept(deptno, dname, loc)
2 VALUES (deptno_seq.NEXTVAL, 'SUPPORT', 'PUNE');
```

1 row created.

```
SQL> SELECT * FROM DEPT;
```

DEPTNO	DNAME	LOC
10	ACCOUNTING	NEW YORK
20	RESEARCH	DALLAS
30	SALES	CHICAGO
40	OPERATIONS	BOSTON
50	PERSONAL	MUMBAI
60	SUPPORT	PUNE

6 rows selected.

```
SQL> SELECT deptno_seq.CURRVAL FROM dual;
```

CURRVAL

60

Modifying / Altering a Sequence

- A sequence is usually altered when it is desirable
 - to set or eliminate the values of the MINVALUE or MAXVALUE parameters,
 - or to change the INCREMENT BY value,
 - or to change the number of cached sequence numbers.
- The **ALTER SEQUENCE** command shown here changes the MAXVALUE of the **rollno_seq** to **19460**.

```
ALTER SEQUENCE rollno_seq MAXVALUE 19460;
```

Sequence altered.

```
ALTER SEQUENCE deptno_seq  
INCREMENT BY 20  
MAXVALUE 999  
NOCYCLE  
NOCACHE;
```

Sequence altered.

Modifying / Altering a Sequence – Guidelines

- You must be the owner or have the `ALTER` privilege for the sequence.
- Only future sequence numbers are affected.
- The sequence must be dropped and re-created to restart the sequence at a different number.
- Some validation is performed.
- When specifying a `MINVALUE` clause, the specified value should be less than the `MAXVALUE` where a sequence generates ascending numbers.
- In the case of a descending sequence, the `MAXVALUE` should be less than the `MINVALUE`.

Caching Sequence Values

- Caching sequence values in memory gives faster access to those values.
- Gaps in sequence values can occur when:
 - A rollback occurs
 - The system crashes
 - A sequence is used in another table

Viewing Sequence Properties/Parameters

- We may need to review the names and properties of our sequences.
- We can do this by querying the **USER_SEQUENCES** system view with a SELECT command. This view is part of the database's data dictionary.

SELECT * FROM USER_SEQUENCES;

```
SQL> SELECT * FROM USER_SEQUENCES;
```

SEQUENCE_NAME	MIN_VALUE	MAX_VALUE	INCREMENT_BY	C	O	CACHE_SIZE	LAST_NUMBER
ROLLNO_SEQ	19401	19432	1	N	N	50	19433

```
SQL> ALTER SEQUENCE rollno_seq MAXVALUE 19460;
```

Sequence altered.

```
SQL> SELECT * FROM USER_SEQUENCES;
```

SEQUENCE_NAME	MIN_VALUE	MAX_VALUE	INCREMENT_BY	C	O	CACHE_SIZE	LAST_NUMBER
ROLLNO_SEQ	19401	19460	1	N	N	50	19406

Viewing Sequence Properties/Parameters

- **DROP SEQUENCE** command is used to drop sequences that need to be recreated or are no longer needed.

- **Syntax:**

DROP SEQUENCE <sequence name>;

- **Example:**

DROP SEQUENCE rollno_seq;

Sequence dropped.

A large, stylized graphic featuring the words "Question & Answer" in a bold, sans-serif font. The word "Question" is in blue, and the ampersand "&" is in yellow. The word "Answer" is also in blue. The text is centered within a large, horizontal oval shape. The oval has a thick yellow border with a blue outline. The background is white.

Question & Answer